Université Abdelmalek Essaâdi
Faculté des Sciences de Tétouan



Licence – semestre 5

Ingénierie Informatique et Développement Logicie (2IDL)

Modélisation statistique et analyse de données

Chapitre 2

Python pour la science des données

Année Universitaire : 2026/2027

Prof. Ouissam ZEALOUK

PLAN

- Introduction et Concepts Fondamentaux de Python
- Structures de Données Python
- Logique et Contrôle de Flux en Python
- Manipulation des Données en Python

Introduction

- Python est un langage de programmation simple, clair et interprété, qui permet de développer rapidement des applications dans divers domaines. Polyvalent et puissant, il convient aussi bien aux débutants qu'aux programmeurs expérimentés.
- Domaines d'application : Développement web, science des données, apprentissage automatique (machine learning), automatisation de tâches, intelligence artificielle, traitement du langage naturel, développement de jeux.
- syntaxe simple et lisible facilite l'écriture, la compréhension et la maintenance du code, rendant le développement plus rapide et plus efficace.

❑ Types de Données et Variables

- **Concepts :** Types de données (int, float, str, bool, None), déclaration et utilisation de variables

Exemples :

```
entier = 10                 # Type int, stocke un nombre entier
flottant = 3.14159          # Type float, pour des nombres à virgule
texte = "Bonjour Python"    # Type str, pour une chaîne de caractères
est_actif = True            # Type bool, représente une valeur de vrai/faux
```

- ✓ Variables : En Python, une variable est un conteneur pour stocker une valeur. Le nom de la variable est comme une étiquette que l'on peut réutiliser pour accéder à la valeur.
- ✓ None : Python utilise None pour représenter l'absence de valeur. Cela peut être utile pour indiquer qu'une variable n'a pas encore été initialisée.

❑ Types de Données et Variables

- **La fonction type()** en Python permet de vérifier le type d'une variable ou d'un objet. Par exemple, si vous avez une variable x et que vous souhaitez connaître son type, vous pouvez utiliser la commande **type(x)**. Cela renverra le type de x, comme int pour un entier, str pour une chaîne de caractères

Exemples :

```
>>> x = 10
>>> type(x)
<class 'int'>

>>> pi = 3.14159
>>> type(pi)
<class 'float'>

>>> msg= 'Bonjour python'
>>> type(msg)
<class 'str'>

>>> est_actif = true
>>> type(est_actif )
<class 'bool'>
```

❑ Types de Données et Variables

Les constantes

les constantes sont des variables dont la valeur est censée rester fixe pendant l'exécution du programme. Python ne propose pas de mot-clé spécifique pour créer une constante, mais il est courant d'utiliser des lettres majuscules pour le nom de la variable afin d'indiquer qu'elle ne doit pas être modifiée.

Exemples :

```
SECONDS_PER_MINUTE = 60          # Nombre de secondes dans une minute

VITESSE_LUMIERE = 299792458      # Vitesse de la lumière en mètres par seconde
```

❑ Types de Données et Variables

conversion de type (transtypage)

- Le transtypage en Python consiste à changer le type d'une valeur pour un autre type compatible.

Exemples : >>> float (5) 5.0 >>> int (7.9) 7 >>> str (123) ' 123 '

- La conversion explicite, également appelée "casting" en Python, permet de transformer volontairement une valeur d'un type de donnée en un autre afin de l'utiliser dans un contexte spécifique.

Exemples :

Flottant à Entier

```
>>> x = 7.9
>>> y = int(x)
# y devient 7
```



Certains conversions de type ne sont pas autorisés

```
>>> x = "Bonjour"
>>> y = int(x)
```

❑ Types de Données et Variables

Exemple

```
# Variable booléenne pour représenter l'état de connexion
est_connecte = True

nom = "Aziz"

# Vérification de l'état
if est_connecte:
    print(f"{nom} est connecté.")
else:
    print(f"{nom} n'est pas connecté.")
```

Le préfixe f avant les guillemets indique que nous utilisons une f-string. Cela permet d'insérer directement des expressions dans la chaîne en les encadrant avec des accolades {}

❑ Utilisation des données en Python « Lecture de données »

- **Concepts** :La fonction input() permet de lire une donnée saisie par l'utilisateur au clavier. Elle peut afficher un message pour indiquer ce que l'utilisateur doit entrer.
- La valeur saisie est toujours récupérée sous forme de str (string), quel que soit le type de donnée attendu.
- Cette donnée peut ensuite être stockée dans une variable et utilisée dans des traitements, des calculs ou des affichages à l'aide d'autres fonctions Python.

```
nom = input("Entrez votre nom : ")
print(f"Bienvenue, {nom}!")
```

❑ Utilisation des données en Python « Lecture de données »

- **Lecture d'une donnée numérique**

- Pour lire une donnée de type numérique, on utilise les fonctions prédéfinies int(), float() pour convertir la donnée lue à l'aide de la fonction input().

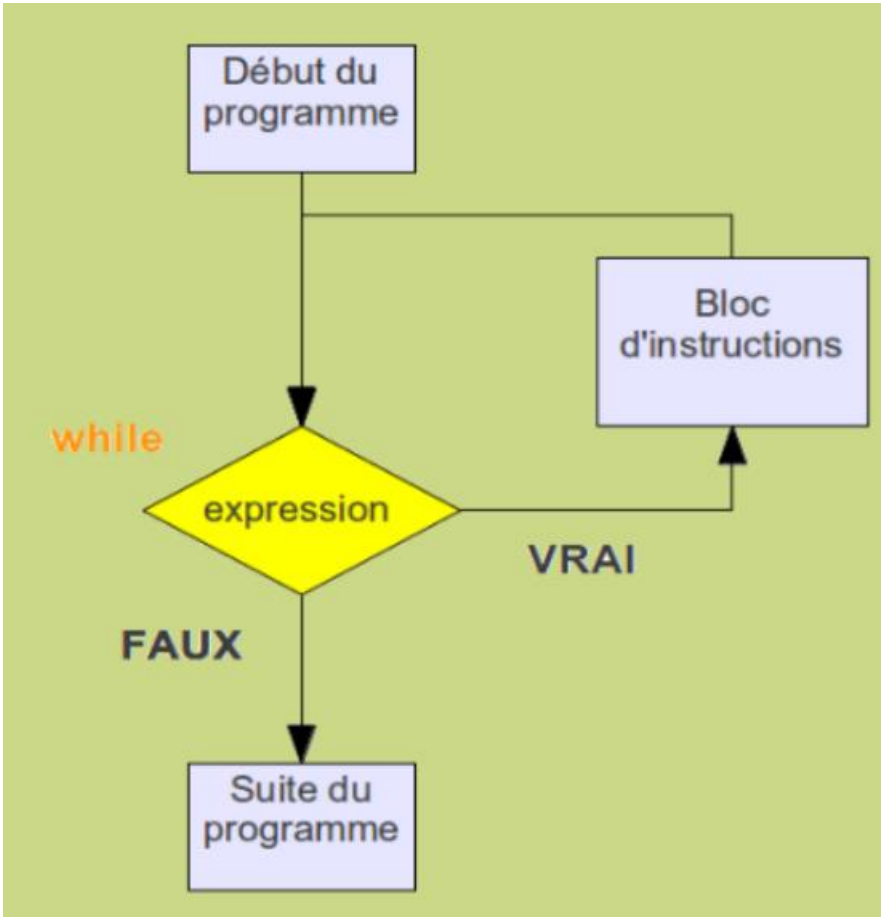
```
largeur = int(input("Entrez la largeur : "))
Entrez la largeur : 10
>>> hauteur = float(input("Entrez la hauteur : "))
Entrez la hauteur : 6.5
print(largeur * hauteur)
65.0
```

❑ Contrôle du flux d'exécution : Instructions répétitives (boucle) :

➢ Boucle while (Tant que)

La boucle **while** permet d'exécuter un bloc de code tant qu'une condition (une expression retournant une valeur booléenne True ou False) est vérifiée. Cette structure de contrôle est essentielle pour réaliser des tâches répétitives sans avoir à dupliquer le code. Lorsque la condition devient fausse, le programme passe à l'instruction suivante, permettant ainsi une gestion fluide du flux d'exécution.

```
while condition:
    # bloc de code à exécuter tant que la condition est vraie
```



❑ Contrôle du flux d'exécution : Instructions répétitives (boucle) :

➢ Boucle while (Tant que)

Au démarrage de la boucle, Python évalue la condition spécifiée. Si elle est vraie, le corps de la boucle est exécuté, et ce processus se répète jusqu'à ce qu'elle change. Si la condition est fausse, le bloc d'instructions associé est ignoré, et l'exécution continue.

```
a = 0
while a < 7:
    print(a)
    a += 1
```



```
0
1
2
3
4
5
6
```

Remarque: sans l'instruction a +=1, la boucle ne s'arrête jamais:

Initialisation : La variable a est initialisée à 0.

Condition : La boucle while continue tant que a est inférieur à 7.

Mise à jour : À chaque itération, a est incrémenté de 1.

❑ Contrôle du flux d'exécution : Instructions répétitives (boucle) :

➢ Boucle while (Tant que)

```
compteur = 0
while compteur < 7:
    print(f"{compteur} {compteur < 7}")
    compteur += 1
print("fin de la boucle")
```

```
0 True
1 True
2 True
3 True
4 True
5 True
6 True
fin de la boucle
```

```
compteur = 0
while compteur < 7:
    compteur += 1  $
    print(f"{compteur} {compteur < 7}")
print("fin de la boucle")
```

```
1 True
2 True
3 True
4 True
5 True
6 True
7 False
fin de la boucle
```

❑ Contrôle du flux d'exécution : Instructions répétitives (boucle) :

➢ Boucle for

La boucle for est une structure de contrôle qui permet de répéter un bloc d'instructions un certain nombre de fois en itérant sur une séquence d'éléments. Elle est utilisée pour parcourir les éléments d'un objet itérable, comme une chaîne de caractères ou un ensemble de valeurs, et exécuter les instructions spécifiées pour chaque élément..

```
for element in sequence:
    #bloc de code à exécuter
```

L'élément est une variable introduite par la boucle for, et il n'est pas nécessaire de l'initialiser. Elle prend successivement chacune des valeurs de la séquence parcourue. De plus, l'opérateur in peut être utilisé dans d'autres contextes en dehors des boucles for.

❑ Contrôle du flux d'exécution : Instructions répétitives (boucle) :

➢ Boucle for

```
chaine = "FS Tetouan"
for lettre in chaine:
    print(lettre)
```

```
F
S

T
e
t
o
u
a
n
```

La varaible lettre est initialisée avec le premier élément de la séquence "F"

Le bloc d'instruction est alors exécuté

Puis la variable lettre est mise à jour avec le seconde élément de la séquence "S" et le bloc d'instruction à nouveau exécuté....

Le bloc d'instruction est exécuté une dernière fois lorsqu'on arrive au dernier élément de la de séquence "n"

❑ Contrôle du flux d'exécution : Instructions répétitives (boucle) :

➢ Boucle for

la fonction range() est utilisée pour générer une séquence de nombres. Elle peut prendre un, deux ou trois arguments :

```
for i in range(5):
    print(i)
```

```
0
1
2
3
4
```

```
for i in range(2, 5)
    print(i)
```

```
2
3
4
```

```
for i in range(2, 12, 3):
    print(i)
```

```
2
5
8
11
```

❑ Contrôle du flux d'exécution : Instructions répétitives (boucle) :

➢ Boucle for

Exercice

Écris un programme qui affiche les carrés des nombres de 1 à 10

```
for i in range(1, 11):
    carre = i ** 2
    print(f"Le carré de {i} est {carre}")
```

```
Le carré de 1 est 1
Le carré de 2 est 4
Le carré de 3 est 9
Le carré de 4 est 16
Le carré de 5 est 25
Le carré de 6 est 36
Le carré de 7 est 49
Le carré de 8 est 64
Le carré de 9 est 81
Le carré de 10 est 100
```

❑ Contrôle du flux d'exécution : Instructions répétitives (boucle) :

➢ Boucle for

L'instruction break est utilisée pour interrompre l'exécution d'une boucle (comme une boucle for ou while) lorsqu'une certaine condition est remplie. Cela permet de sortir de la boucle immédiatement, sans attendre que toutes les itérations soient terminées

```
for i in range(1, 10):
    print("debut",i)
    if i == 6:
        break  # Sort de la boucle lorsque i est égal à 6
    print("fin",i)
```

```
debut 1
fin 1
debut 2
fin 2
debut 3
fin 3
debut 4
fin 4
debut 5
fin 5
debut 6
```

❑ Contrôle du flux d'exécution : Instructions répétitives (boucle) :

➢ Boucle for

L'instruction continue permet de sauter l'itération en cours d'une boucle et de passer directement à la prochaine itération. Elle est souvent utilisée pour ignorer certaines conditions sans sortir complètement de la boucle

```
for i in range(5):
    if i == 2:
        continue  # Ignore l'itération quand i vaut 2
    print(i)
```

```
0
1
3
4
```

Structures de données :

Les structures de données sont des moyens d'organiser et de stocker des données de manière à ce qu'elles puissent être utilisées efficacement. En Python, les structures de données intégrées permettent de stocker, organiser et manipuler des collections d'éléments de différentes manières

- ***Les listes***
- ***Les tuples***
- ***Les Dictionnaires***
- ***les ensembles***

Structures de données :

Les Listes

- Une liste est une structure de données qui contient une série de valeurs de types différents (par exemple entier et chaîne de caractères), ce qui leur donne une grande flexibilité.
- Une liste est déclarée par une série de valeurs séparées par des virgules, et le tout est encadré par des crochets [].

```
taille = [165,170,180,168.5,175]

etudiants = ["Ahmed","Karim","Fatima","salwa"]
```

- Vous pouvez voir le contenu de la liste **taille** et **etudiants** en l'appelant comme ceci:

```
taille
[165, 170, 180, 168.5, 175]
etudiants
['Ahmed', 'Karim', 'Fatima', 'salwa']
```

Ou bien

```
print (taille)
[165, 170, 180, 168.5, 170, 175]
print(etudiants)
['Ahmed', 'Karim', 'Fatima', 'salwa']
```

Structures de données :

Les Listes

- Les listes **a** et **b** constituée d'objets de types hétérogènes

```
a = [42, "Bonjour", 3.14, "python"]
```

La liste **a** contient 4 éléments

```
b = [0, 1.5 , "abc" , 2024, "Ok"]
```

La liste **b** contient 5 éléments.

- Une liste peut tout à fait contenir d'autres listes en tant qu'éléments, permettant ainsi de créer des structures de données imbriquées, comme des matrices.

```
c = [[], [1], [1,2], [1, 2, 3]]
```

La liste **c** contient 4 éléments

Structures de données :

Les Listes

- On peut appeler un élément d'une liste par un numéro indiquant sa position, nommé indice (ou index) de la liste.
- les indices d'une liste de n éléments commencent à 0 et se terminent à n-1.

```
taille = [165,170,180,168.5,175]

etudiants = ["Ahmed","Karim","Fatima","salwa"]

a = [42, "Bonjour", 3.14, "python"]

taille
[165, 170, 180, 168.5, 175]
etudiants
['Ahmed', 'Karim', 'Fatima', 'salwa']
a
[42, 'Bonjour', 3.14, 'python']
```

```
taille[0]
165

taille[4]
175

etudiants[2]
'Fatima'

a[4]
Traceback (most recent call last):
  File "<pyshell#6>", line 1, in <module>
    a[4]
IndexError: list index out of range
```

Structures de données :

Les Listes

- Pour accéder à un élément à l'intérieur d'une liste qui est elle-même contenue dans une autre liste (liste imbriquée), on utilise des indices multiples, séparés par des crochets. Par exemple c :

```
b = [0, 1.5 , "abc" , 2024, "Ok"]

c = [[], [1], [1,2], [1, 2, 3]]


b
[0, 1.5, 'abc', 2024, 'Ok']
c
[[], [1], [1, 2], [1, 2, 3]]
```

```
b[1]
1.5

c[1]
[1]

c[2]
[1, 2]

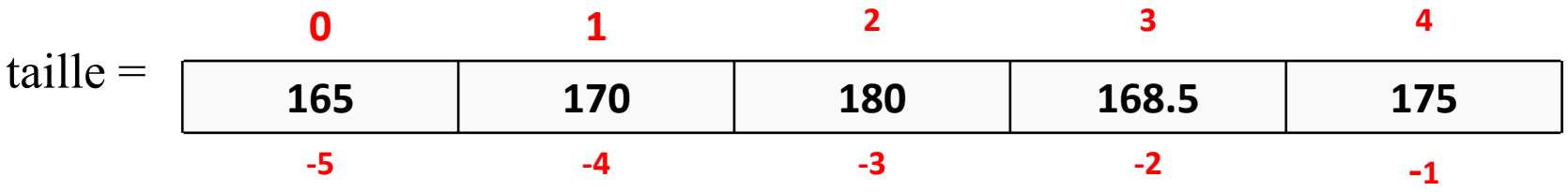
c[3][2]
3

c[3][3]
Traceback (most recent call last):
  File "<pyshell#17>", line 1, in <module>
    c[3][3]
IndexError: list index out of range
```

Structures de données :

Les Listes

- La liste peut également être indexée avec des nombres négatifs
- On accède au dernier élément d'une liste à l'aide de l'indice -1, l'avant-dernier élément par l'indice -2, etc.




```
taille[-1]
175
```

```
taille[-2]
168.5
```

```
taille[-5]
165
```

Structures de données :

Les Listes

- On utilise l'opérateur + pour la concaténation et l'opérateur * pour la duplication.

```
a = [42, "Bonjour", 3.14, "python"]

b = [0, 1.5 , "abc" , 2024, "Ok"]


x=a+b
x
[42, 'Bonjour', 3.14, 'python', 0, 1.5, 'abc', 2024, 'Ok']

y=a*3
y
[42, 'Bonjour', 3.14, 'python', 42, 'Bonjour', 3.14, 'python', 42,
'Bonjour', 3.14, 'python']
```

Structures de données :

Les Listes

- La méthode extend permet d'ajouter tous les éléments d'une liste à la fin d'une autre liste. Contrairement à l'opérateur +, qui crée une nouvelle liste, extend modifie directement la liste originale.

Dans cet exemple, extend ajoute les éléments de liste b à liste a. Après l'appel à extend, liste a contient les éléments des deux listes.

```
a = [42, "Bonjour", 3.14, "python"]

b = [0, 1.5 , "abc" , 2024, "Ok"]


a.extend(b)
a
[42, 'Bonjour', 3.14, 'python', 0, 1.5, 'abc', 2024, 'Ok']
```

Structures de données :

Les Listes

- La longueur d'une liste, c'est-à-dire le nombre d'éléments que contient la liste est obtenue par la fonction len().

```
a = [42, "Bonjour", 3.14, "python"]

len(a)
4

x=a*2
x
[42, 'Bonjour', 3.14, 'python', 42, 'Bonjour', 3.14, 'python']

len(x)
8
```

Structures de données :

Les Listes (ajouter un élément à la fin d'une liste)

- La méthode .append() est utilisée pour ajouter un élément à la fin d'une liste. Cette méthode modifie la liste en place, c'est-à-dire qu'elle n'a pas besoin de créer une nouvelle liste pour ajouter un élément. Elle est très utile lorsqu'on veut construire ou étendre une liste de manière dynamique.

En utilisant +

```
a
[42, 'Bonjour', 3.14, 'python']

a+[10]
[42, 'Bonjour', 3.14, 'python', 10]

a
[42, 'Bonjour', 3.14, 'python']
```

En utilisant .append()

```
a
[42, 'Bonjour', 3.14, 'python']

a.append(10)

a
[42, 'Bonjour', 3.14, 'python', 10]
```

Structures de données :

Les Listes (Ajouter un élément à une position précise dans une liste)

- Pour insérer un élément dans une liste à une position spécifique. Contrairement à des méthodes comme append(), qui ajoute un élément à la fin de la liste, insert() vous permet de spécifier l'indice où vous souhaitez ajouter l'élément.

```
a = [42, "Bonjour", 3.14, "python"]

a
[42, 'Bonjour', 3.14, 'python']

a.insert(2,100)

a
[42, 'Bonjour', 100, 3.14, 'python']
```

Structures de données :

Les Listes

- Pour modifier la valeur d'un élément dans une liste en accédant à son index et en lui affectant une nouvelle valeur, Par exemple, a[3] = "IME/PA" remplace l'élément à l'index 3 de la liste par "IME/PA".

```
a = [42, "Bonjour", 3.14, "python"]

a[3]
'python'

a[3]="IME/PA"

a
[42, 'Bonjour', 3.14, 'IME/PA']
```

Structures de données :

Les Listes (Trouver un item dans une liste)

- Pour vérifier si un élément est présent dans une liste, on peut utiliser le mot-clé in, qui permet de déterminer si une valeur donnée existe dans une séquence.

```
taille = [165, 170, 180, 168.5, 170, 175]

taille
[165, 170, 180, 168.5, 170, 175]

180 in taille
True

190 in taille
False
```

Structures de données :

Les Listes (Retirer un élément ou la liste complète)

- Il est parfois nécessaire de supprimer une entrée d'une liste, et pour cela, on peut utiliser la fonction del, qui permet de retirer un élément spécifique à partir de son index.

```
b = [0, 1.5 , "abc" , 2024, "Ok"]

b
[0, 1.5, 'abc', 2024, 'Ok']

del b[2]

b
[0, 1.5, 2024, 'Ok']
```

- Avec del, il est aussi possible de supprimer une tranche d'une liste, c'est-à-dire plusieurs éléments à la fois

```
b = [0, 1.5 , "abc" , 2024, "Ok"]

b
[0, 1.5, 'abc', 2024, 'Ok']

del b[1:4]

b
[0, 'Ok']
```

Structures de données :

Les Listes (Retirer un élément ou la liste complète)

- Enfin, pour supprimer complètement une liste (la variable et son contenu sont définitivement supprimés), on peut utiliser del sur la liste elle-même.

```
b = [0, 1.5 , "abc" , 2024, "Ok"]
b
[0, 1.5, 'abc', 2024, 'Ok']

del b
b
NameError: name 'b' is not defined
```

- Contrairement à del, la méthode clear() vide le contenu de la liste, mais la liste elle-même reste en mémoire et devient simplement vide.

```
b = [0, 1.5 , "abc" , 2024, "Ok"]
b
[0, 1.5, 'abc', 2024, 'Ok']

b.clear()
b
[]
```

Structures de données :

Les Listes (Retirer un élément ou la liste complète)

- Pour supprimer un élément d'une liste en fonction de sa valeur, on utilise la méthode remove() plutôt que del. La méthode remove() supprime la première occurrence de la valeur spécifiée dans la liste

```
taille = [165, 170, 180, 168.5, 170, 175]

taille
[165, 170, 180, 168.5, 170, 175]

taille.remove(180)

taille
[165, 170, 168.5, 170, 175]
```

```
taille.remove(170)

taille
[165, 168.5, 170, 175]
```

Pour supprimer la deuxième occurrence de 170, vous devrez appeler remove(170) une seconde fois

```
taille
[165, 168.5, 175]
```

Structures de données :

Les Listes (Retirer un élément ou la liste complète)

- Pour supprimer toutes les occurrences de 170 dans la liste:

```
taille = [165, 170, 170, 180, 168.5, 170, 175, 170]
```

On peut utiliser une boucle while qui continue à supprimer 170 tant qu'il reste dans la liste, afin d'éviter des problèmes liés à l'itération et à la modification simultanées de la liste

```
taille
[165, 170, 170, 180, 168.5, 170, 175, 170]
while 170 in taille:
    taille.remove(170)

print(taille)
[165, 180, 168.5, 175]
```

Structures de données :

Les Listes (la position d'un élément dans une liste)

- La méthode index() vous permet de connaitre la position de l'item cherché

```
taille = [165, 170, 170, 180, 168.5, 170, 175, 170]
```

```
taille.index(180)
3
```

- Si l'élément apparaît plusieurs fois dans la liste, index() renvoie l'index de la première occurrence.

```
taille.index(170)
1
```

- Si l'élément n'existe pas dans la liste, la méthode lève une exception ValueError

```
taille.index(190)
ValueError: 190 is not in list
```

Structures de données :

Les Listes (Compter les occurrences dans une liste)

- la méthode count() permet de compter le nombre d'occurrences d'un élément spécifique dans une liste.

```
taille = [165, 170, 170, 180, 168.5, 170, 175, 170]
```

Si vous voulez savoir combien de fois l'élément 170 apparaît dans la liste taille, vous utilisez les deux approches suivantes :

```
element = 170
nombre_occurences = taille.count(element)
print("nombre_occurences de 170 est {} ".format(nombre_occurences))
```

```
nombre_occurences de 170 est 4
```

```
print(taille.count(170))
```

```
4
```

```
print(taille.count(180))
```

```
1
```

Structures de données :

Les Listes

- Pour avoir le minimum, le maximum et la somme des éléments d'une liste, on utilise respectivement les fonctions intégrées min(), max() et sum(). Ces fonctions sont des outils essentiels pour effectuer des opérations simples mais puissantes sur les listes.

```
taille = [165, 170, 170, 180, 168.5, 170, 175, 170]

max(taille)
180

min(taille)
165

sum(taille)
1368.5
```

```
a = [42, "Bonjour", 3.14, "python"]

max(a)
```

```
TypeError: '>' not supported between instances of 'str' and 'int'
```

Ne fonctionne pas directement car les éléments de la liste sont de types différents (nombres, chaînes de caractères, etc.).

Structures de données :

Tri d'une Liste

- La méthode .sort() trie les éléments d'une liste du plus petit au plus grand.
- La méthode sorted() crée une nouvelle liste triée, sans modifier la liste d'origine

```
liste1 = [8, 5, 7, 2, 4]
```

```
liste1.sort()
liste1
[2, 4, 5, 7, 8]
```

```
liste2 = list(sorted(liste1))
print(liste2)
[2, 4, 5, 7, 8]
print(liste1)
[8, 5, 7, 2, 4]
```

Structures de données :

Inverse d'une Liste

- La méthode .reverse() inverse une liste..
- La méthode reversed() crée une nouvelle liste inversée, sans modifier la liste d'origine

```
liste1 = [8, 5, 7, 2, 4]
```

```
liste1
[8, 5, 7, 2, 4]

liste1.reverse()
liste1
[4, 2, 7, 5, 8]
```

```
liste2 = list(reversed(liste1))
print(liste2)
[4, 2, 7, 5, 8]
print(liste1)
[8, 5, 7, 2, 4]
```

Structures de données :

Tri dans l'ordre décroissant d'une Liste

- L'argument reverse=True spécifie le tri inverse, c'est-à-dire du plus grand au plus petit décroissant.

```
liste1 = [8, 5, 7, 2, 4]
liste1
[8, 5, 7, 2, 4]
liste1.sort(reverse=True)
liste1
[8, 7, 5, 4, 2]
```

- L'instruction liste2 = sorted(liste1, reverse=True) crée une nouvelle liste triée dans l'ordre décroissant sans modifier la liste d'origine.

```
liste2= sorted(liste1, reverse=True)
liste2
[8, 7, 5, 4, 2]
```

Structures de données :

Copy une Liste

En Python, lorsqu'on effectue une affectation d'une liste à une autre, comme dans l'exemple suivant:

```
a = [42, "Bonjour", 3.14, "python"]
y=a
```

y = a **ne crée pas une nouvelle copie de la liste**, mais fait en sorte que y et a pointent vers la même liste en mémoire. Les deux variables a et y font référence à l'objet original en mémoire.

Cela signifie que si tu modifies l'une des listes, l'autre sera également modifiée, car elles partagent le même espace mémoire.

```
y=a
a.append(100)
a
[42, 'Bonjour', 3.14, 'python', 100]
y
[42, 'Bonjour', 3.14, 'python', 100]
```

Structures de données :

Copy une Liste

Pour créer une **copie superficielle** de la liste, ce qui signifie qu'elle crée une nouvelle liste avec les mêmes références aux objets que la liste originale, on utilise la méthode copy() ou bien la méthode list(). (En cas de modifications des éléments, cela n'affectera pas l'original, sauf si les éléments eux-mêmes sont des objets mutables).

```
a = [42, "Bonjour", 3.14, "python"]
```

```
a
[42, 'Bonjour', 3.14, 'python']
y=a.copy()
y
[42, 'Bonjour', 3.14, 'python']
y.append(100)
y
[42, 'Bonjour', 3.14, 'python', 100]
a
[42, 'Bonjour', 3.14, 'python']
```

```
a
[42, 'Bonjour', 3.14, 'python']
y=list(a)
y
[42, 'Bonjour', 3.14, 'python']
y.append(100)
y
[42, 'Bonjour', 3.14, 'python', 100]
a
[42, 'Bonjour', 3.14, 'python']
```

Structures de données :

Copy une Liste

Pour manipuler des données plus complexes, telles que des listes imbriquées, il est souvent nécessaire de créer une copie indépendante de la structure originale. Dans ce cas, la fonction deepcopy() du module copy est particulièrement utile.

```
import copy

c = [[], [1], [1,2], [1, 2, 3]]
y = copy.deepcopy(c)
y[3][1] = 99

y
[[], [1], [1, 2], [1, 99, 3]]
c
[[], [1], [1, 2], [1, 2, 3]]
```

Structures de données :

boucle sur les Listes

- En utilisant la boucle for avec la fonction range(), il est possible d'itérer sur les indices d'une liste, ce qui permet d'accéder facilement aux éléments en fonction de leur position dans la séquence."

```
etudiants = ['Ahmed', 'Karim', 'Fatima', 'salwa']
for i in range(4):
    print("l'etudiant",i," est:" ,etudiants[i])
print("Au revoir")
```

Ou bien

```
etudiants = ['Ahmed', 'Karim', 'Fatima', 'salwa']
for i in range(len(etudiants)):
    print("l'etudiant",i," est:" ,etudiants[i])
print("Au revoir")
```

```
l'etudiant 0  est: Ahmed
l'etudiant 1  est: Karim
l'etudiant 2  est: Fatima
l'etudiant 3  est: salwa
Au revoir
```

Structures de données :

Les tuples

- Un tuple est une structure de données qui permet de regrouper un ensemble d'éléments. Il est similaire à une liste, mais avec une différence: un tuple est immuable. Cela signifie que, une fois créé, ses éléments ne peuvent ni être modifiés, ni ajoutés, ni supprimés.

```
a = (1, 50.5 ,'IME', 'PA')

b = (3, 5, 2, 4, 9)
```

- Pour afficher le contenu d'un tuple, cela se fait de la même manière que pour une liste.

```
a
(1, 50, 'IME', 'PA')
b
(8, 5, 7, 2, 4)
```

Ou bien

```
print(a)
(1, 50, 'IME', 'PA')
print(b)
(8, 5, 7, 2, 4)
```

Structures de données :

Les tuples

- Les parenthèses ne sont pas obligatoires mais facilite la lisibilité du code (rappelons que la force de python est sa simplicité de lecture):

```
c = 5, 8, 20, 100
```

- Après l'exécution: `print(type(c))`

⟹ il me renvoie `<class 'tuple'>` ce qui indique que c est un objet de type tuple."

- Un "tuple vide" est un tuple qui ne contient aucun élément. Il est représenté par des parenthèses vides ():

```
d =()
print(d)
()
```

Structures de données :

Les tuples

- Lorsqu'on crée un tuple contenant un seul élément, il est nécessaire d'utiliser une virgule à la fin pour distinguer ce tuple d'une simple valeur entre parenthèses.

```
tuple = (35)
print(type(tuple))
<class 'int'>
tuple = (35,)
print(type(tuple))
<class 'tuple'>
```

Structures de données :

Les tuples

- l'indexation fonctionne de la même manière qu'avec une liste, en utilisant des indices numériques, qui commencent à 0 et vous pouvez également utiliser des indices négatifs pour accéder aux éléments en partant de la fin de la séquence.

```
a = (1, 50.5 ,'IME', 'PA')

b = (3, 5, 2, 4, 9)
```

```
a[2]
'IME'

b[4]
9
```

- Un tuple peut contenir d'autres tuples comme éléments, créant ainsi un "tuple imbriqué".

```
tuple = (8, (5,10), 7, (2,4,6), 4)
```

```
tuple[1]
(5, 10)
tuple[3][1]
4
```

Structures de données :

Les tuples

- La longueur d'une typle, c'est-à-dire le nombre d'éléments que contient la liste est obtenue par la fonction len().

```
a = (1, 50.5 ,'IME', 'PA')

b = (3, 5, 2, 4, 9)

len(a)
4
len(b)
5
```

- Vérifier la présence d'un élément dans le tuple avec in :

```
print(50.5 in a)
True
print(20 in a)
False
```

Structures de données :

Les tuples

- Les éléments d'un tuple sont immuables, ce qui signifie qu'une fois créé, il n'est pas possible de modifier, ajouter ou supprimer ses éléments.

```
print(a)
(1, 50.5, 'IME', 'PA')
a[3] = 10
TypeError: 'tuple' object does not support item assignment
```

- Pour effectuer une concaténation (+) ou une duplication (*), il est toujours nécessaire de créer un nouveau tuple.

```
tuple=a+b
tuple
(1, 50.5, 'IME', 'PA', 3, 5, 2, 4, 9)
```

```
tuple=a*2
tuple
(1, 50.5, 'IME', 'PA', 1, 50.5, 'IME', 'PA')
```

Structures de données :

Les tuples

- Bien que les tuples soient immuables, tu peux créer un nouveau tuple en combinant l'ancien tuple avec de nouveaux éléments :

```
a = (1, 50.5 ,'IME', 'PA')
tuple = a + (100,)
tuple
(1, 50.5, 'IME', 'PA', 100)
mon_tuple = tuple + ('python', 'S3')
mon_tuple
(1, 50.5, 'IME', 'PA', 100, 'python', 'S3')
```

- les méthodes append() et extend() n'existent pas pour les tuples

```
a.append(100)
a.append(b)
```

AttributeError: 'tuple' object has no attribute 'append'

Structures de données :

Les tuples (Affectation multiple - décomposition des tuples)

- l'affectation multiple avec des tuples fonctionne parfaitement car Python "dépaquette" les éléments du tuple de droite et les assigne à chaque variable correspondante à gauche. Cela ne modifie pas le tuple lui-même, mais simplement les variables auxquelles il est assigné.

```
tuple1 = (5, 8, 20, 100)
x,y,z,t = tuple1
print(x)
5
print(y)
8
print(z)
20
print(t)
100
```

```
tuple2 = (5, 8)
x,y,z = tuple2
```

❗ Si vous affectier un tuple avec un nombre d'éléments différent du nombre de variables

⬇

une erreur de type ValueError sera levée

```
ValueError: not enough values to unpack (expected 3, got 2)
```

Structures de données :

Les tuples (Affectation avec un nombre variable d'éléments)

- Si tu as un tuple avec plusieurs éléments et que tu ne veux capturer que certains d'entre eux, tu peux utiliser * pour "attraper" le reste

```
tuple1 = (5, 8, 20, 100)
x,y,*rest = tuple1
print(x)
5
print(y)
8
print(rest)
[20, 100]
```

Ici, x et y sont assignés aux premières valeurs, et rest devient une liste contenant les valeurs

Structures de données :

Les tuples (Transformation d'un Tuple en Liste)

- La transformation d'un tuple en liste est utile et se fait facilement grâce à la fonction list(), tandis que la transformation d'une liste en tuple se fait avec la fonction tuple(). Ces fonctions prennent respectivement un tuple ou une liste en entrée et renvoient une nouvelle liste ou un nouveau tuple contenant les mêmes éléments.

```
tuple1 = (5, 8, 20, 100)
x = list(tuple1)
print(x)
[5, 8, 20, 100]
```

```
liste1 = [8, 5, 7, 2, 4]
x = tuple(liste1)
x
(8, 5, 7, 2, 4)
```

Structures de données : 09/10/2026

Les Dictionnaires

- Un dictionnaire est une structure de données utilisée pour stocker des paires clé-valeur. Chaque élément d'un dictionnaire est une association entre une clé unique et une valeur correspondante. Les dictionnaires sont définis en utilisant des accolades {}, avec chaque paire clé-valeur sép

```
dictionnaire = {"nom": "ahmed", "âge": 20, "ville": "tetouan"}
print(dictionnaire)
{'nom': 'ahmed', 'âge': 20, 'ville': 'tetouan'}
```

- Contrairement aux séquences (listes, tuples), qui sont des suites ordonnées d'éléments, les dictionnaires ne sont pas des séquences.

Structures de données :

Les Dictionnaires

- Dans une séquence, on accède facilement à un élément via un index (un nombre entier), mais il est nécessaire de connaître l'emplacement de l'élément.
- Dans un dictionnaire, on peut accéder à n'importe quel élément en utilisant une clé spécifique, qui peut être alphabétique, numérique, ou même un type composite, sous certaines conditions. ***Par exemple***, pour afficher la valeur associée à la clé 'nom' dans le dictionnaire, il suffit d'utiliser cette clé comme indice.

```
print(dictionnaire["nom"])
ahmed
```

- Les dictionnaires sont modifiables comme les listes, mais leurs éléments ne sont pas disposés dans un ordre immuable.

Structures de données :

Les Dictionnaires (Accéder a un élément)

- Afficher le type d'un dictionnaire :

```
dictionnaire = {"nom": "ahmed", "âge": 20, "ville": "tetouan"}
print(type(dictionnaire))
<class 'dict'>
```

- Accéder à un élément par sa clé avec des crochets [].

```
print(dictionnaire["âge"])
20
```

- Accéder à un élément par sa clé avec la méthode .get() :

```
print(dictionnaire.get("ville"))
tetouan
```

[ ] : Si la clé demandée n'existe pas, Python lève une exception **KeyError**

.get() : Si la clé demandée n'existe pas, cela renvoie **None** par défaut

Structures de données :

Les Dictionnaires (Modifier- Ajouter une valeur)

- Pour modifier la valeur d'un élément à une clé donnée dans un dictionnaire, vous utilisez la syntaxe avec des crochets, comme pour les listes.

```
dictionnaire = {"nom": "ahmed", "âge": 20, "ville": "tetouan"}
dictionnaire["âge"] = 26
print(dictionnaire)
{'nom': 'ahmed', 'âge': 26, 'ville': 'tetouan'}
```

- Pour ajouter un élément dans un dictionnaire, vous utilisez la clé entre crochets [clé]

```
dictionnaire['note'] = 18
print(dictionnaire)
{'nom': 'ahmed', 'âge': 20, 'ville': 'tetouan', 'note': 18}
```

- si la clé est déjà existante, il n'y aura pas d'insertion de nouvelle valeur, mais bien un remplacement.

Structures de données :

Les Dictionnaires (Supprimer une valeur et tout le contenu)

- Supprimer une clé et sa valeur par del :

```
dictionnaire = {"nom": "ahmed", "âge": 20, "ville": "tetouan"}
del dictionnaire["nom"]
print(dictionnaire)
{'âge': 20, 'ville': 'tetouan'}
```

- La méthode clear() supprime toutes les paires clé-valeur du dictionnaire, laissant un dictionnaire vide.

```
dictionnaire.clear()

print(dictionnaire)
{}
```

Structures de données :

Les Dictionnaires (Regrouper et Fusionner les Dictionnaires)

- Les éléments d'un dictionnaire ne sont pas disposés dans un ordre particulier. Des opérations comme la concaténation et l'extraction ne peuvent pas s'appliquer au niveau des dictionnaires :

```
dictionnaire= {"nom": "ahmed", "âge": 20, "ville": "tetouan"}
dictionnaire2 = {"module": "info", "S": 3}
dict = dictionnaire + dictionnaire2
TypeError: unsupported operand type(s) for +: 'dict' and 'dict'
```

- D'après la méthode .update(), elle permet d'ajouter les éléments d'un dictionnaire à un autre, en mettant à jour les valeurs des clés existantes ou en ajoutant de nouvelles clés

```
dictionnaire.update(dictionnaire2)
print(dictionnaire)
{'nom': 'ahmed', 'âge': 20, 'ville': 'tetouan', 'module': 'info', 'S': 3}
```

Structures de données :

Les Dictionnaires (Vérifier existence)

- Comme les listes et les tuples, l'instruction in est utilisable aussi avec les dictionnaires. Elle permet de savoir si un dictionnaire comprend une clé bien déterminée.

```
dictionnaire = {"nom": "ahmed", "âge": 20, "ville": "tetouan"}
print("nom"  in dictionnaire)
True
print("filière"  in dictionnaire)
False
```

- On vérifie l'existence d'une valeur par l'utilisation de la collection nom de dictionnaire.values()

```
print(20 in dictionnaire.values())
True
print("tanger" in dictionnaire.values())
False
```

Structures de données :

Les Dictionnaires (les clés , les valeurs et les paires clé-valeur)

- Obtenir toutes les clés ou valeurs .keys() / .values():

```
dictionnaire = {"nom": "ahmed", "âge": 20, "ville": "tetouan"}
```

```
print(dictionnaire.keys())
dict_keys(['nom', 'âge', 'ville'])
```

```
print(dictionnaire.values())
dict_values(['ahmed', 20, 'tetouan'])
```

- Pour parcourir un dictionnaire en Python et obtenir à la fois les clés et les valeurs, utilisez une boucle for avec la méthode .items(), qui retourne chaque paire clé-valeur à chaque itération

```
for clé, valeur in dictionnaire.items():
    print(clé, valeur)
```



```
nom ahmed
âge 20
ville tetouan
```

Structures de données :

Les Dictionnaires (Transformer un Dictionnaire en Liste)

- Pour convertir un dictionnaire en une liste , on peut utiliser la méthode list() sur les clés, valeurs ou paires clé-valeur du dictionnaire, ce qui permet d'obtenir une liste contenant ces éléments selon les besoin:

```
dictionnaire = {"nom": "ahmed", "âge": 20, "ville": "tetouan"}
```

```
liste_cles = list(dictionnaire.keys())
print(liste_cles)
['nom', 'âge', 'ville']
```

```
liste_valeurs = list(dictionnaire.values())
print(liste_valeurs)
['ahmed', 20, 'tetouan']
```

```
liste_paires = list(dictionnaire.items())
print(liste_paires)
[('nom', 'ahmed'), ('âge', 20), ('ville', 'tetouan')]
```

Structures de données :

Les Dictionnaires (Transformer un Dictionnaire en tuple)

- De la même manière, on peut convertir un dictionnaire en un tuple en utilisant la fonction tuple() sur les clés, les valeurs ou les paires clé-valeur.

```
dictionnaire = {"nom": "ahmed", "âge": 20, "ville": "tetouan"}
```

```
liste_cles = tuple(dictionnaire.keys())
print(liste_cles)
('nom', 'âge', 'ville')
```

```
liste_valeurs= tuple(dictionnaire.values())
print(liste_valeurs)
('ahmed', 20, 'tetouan')
```

```
liste_paires = tuple(dictionnaire.items())
print(liste_paires)
(('nom', 'ahmed'), ('âge', 20), ('ville', 'tetouan'))
```

Structures de données :

Les Dictionnaires (Copy une Liste)

- Si le dictionnaire contient uniquement des objets immuables (comme des entiers, des chaînes de caractères), une copie superficielle suffit. Vous pouvez utiliser la méthode copy() ou la fonction dict().

```
dictionnaire = {"nom": "ahmed", "âge": 20, "ville": "tetouan"}
```

```
copie1=dictionnaire.copy()
print(copie1)
{'nom': 'ahmed', 'âge': 20, 'ville': 'tetouan'}
copie1["âge"] = 30
print(copie1)
{'nom': 'ahmed', 'âge': 30, 'ville': 'tetouan'}
print(dictionnaire)
{'nom': 'ahmed', 'âge': 20, 'ville': 'tetouan'}
```

```
copie2=dict(dictionnaire)
print(copie2)
{'nom': 'ahmed', 'âge': 20, 'ville': 'tetouan'}
copie2["âge"] = 30
print(copie2)
{'nom': 'ahmed', 'âge': 30, 'ville': 'tetouan'}
print(dictionnaire)
{'nom': 'ahmed', 'âge': 20, 'ville': 'tetouan'}
```

Structures de données :

Les ensembles

- Un ensemble (ou set en anglais) est une structure de données qui permet de stocker des éléments uniques sans ordre particulier. Cela signifie qu'un ensemble ne conserve pas l'ordre des éléments comme le ferait une liste ou un tuple, et il élimine automatiquement les doublons. Les ensembles en Python sont définis en utilisant des accolades {} ou la fonction set().

```
ensemble_a = {1, 2, 3, 4}

ensemble_b = set(["a", "b", 7, 3])
```

Structures de données :

Les ensembles

- Pour tester le type d'un objet, vous pouvez utiliser la même fonction que celle que nous avons utilisée pour d'autres structures type().

```
ensemble_a = {1, 2, 3, 4}
print(type(ensemble_a))
<class 'set'>
```

- Pour créer un ensemble vide en Python, on utilise set(). Cela permet d'obtenir un ensemble sans éléments, prêt à recevoir des valeurs uniques.

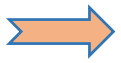
```
ensemble_vide = set()
print(ensemble_vide)
set()
```

Structures de données :

Les ensembles (Affichage des éléments d'un ensemble)

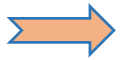
- La **mutabilité** des ensembles les rend similaires aux listes et aux dictionnaires, dans le sens où l'on peut **ajouter** ou **supprimer** des éléments individuels après leur création. Cependant, les ensembles possèdent des caractéristiques spécifiques, telles que le fait qu'ils ne contiennent que des éléments **uniques** (sans doublons) et qu'ils sont **non ordonnés**, ce qui signifie que l'ordre des éléments individuels n'est pas garanti.

```
ensemble_a = {1, 2, 3, 4, 3, 1}
print(ensemble_a)
```



```
{1, 2, 3, 4}
```

```
ensemble_b = set(["a", "b", 7, 3])
print(ensemble_b)
```



```
{'b', 3, 'a', 7}
```

Structures de données :

Les ensembles (Affichage des éléments d'un ensemble)

- Lorsqu'on parcourt un ensemble pour afficher chaque élément individuellement, on utilise généralement une boucle for pour itérer sur l'ensemble. Chaque élément est récupéré un par un. Cependant, l'ordre dans lequel les éléments sont parcourus n'est pas garanti, car les ensembles ne conservent pas l'ordre d'insertion des éléments.

```
ensemble_b = set(["a", "b", 7, 3])
for element in ensemble_b:
    print(element)
```

```
a
3
b
7
```

Structures de données :

Les ensembles (Ajouter un ou plusieurs éléments)

- Pour ajouter un élément à un ensemble, on utilise la méthode add(), qui garantit qu'il n'y a pas de doublon. De même, la méthode update() permet d'ajouter plusieurs éléments à un ensemble en une seule opération.

```
ensemble_a = {1, 2, 3, 4}
ensemble_a.add(100)
print(ensemble_a)
{1, 2, 3, 100, 4}
```

```
ensemble_a = {1, 2, 3, 4}
ensemble_a.add(2)
print(ensemble_a)
{1, 2, 3, 4}
```

Structures de données :

Les ensembles (Ajouter un ou plusieurs éléments)

```
ensemble_b = set(["a", "b", 7, 3])

ensemble_b.update([4, 5, 6])

print(ensemble_b)
{3, 4, 5, 6, 7, 'b', 'a'}
```

```
ensemble_b = set(["a", "b", 7, 3])

ensemble_b.update(["a", 6, 7])

print(ensemble_b)
{3, 6, 7, 'b', 'a'}
```

- Les deux méthodes add() et update() éliminent les redondances. Si un élément est déjà présent dans l'ensemble, il ne sera pas ajouté à nouveau.

Structures de données :

Les ensembles (Supprimer un ou plusieurs éléments)

- Pour supprimer un ou plusieurs éléments spécifiques d'un ensemble, vous pouvez utiliser la méthode remove() à chaque fois.

```
ensemble_b = set(["a", "b", 7, 3])
ensemble_b.remove(7)

print(ensemble_b)
{'b', 3, 'a'}
```

❗ Si l'élément à supprimer n'existe pas, cela va générer erreur **KeyError**

- Si vous devez supprimer plusieurs éléments à la fois, vous pouvez utiliser une boucle comme ceci:

```
elements_supprimer=[7, 3, "a"]
for element in elements_supprimer:
    ensemble_b.remove(element)

print(ensemble_b)
{'b'}
```

Structures de données :

Les ensembles (Vider l'ensemble complètement - Vérifier existence)

- La méthode clear() est utilisée de manière similaire pour vider le contenu des ensembles, des listes et des dictionnaires. Elle supprime tous les éléments de ces collections.

```
ensemble_b.clear()

print(ensemble_b)
set()
```

- Vérifier la présence d'un élément dans l'ensemble ainsi que dans les structures de données précédentes avec l'opérateur in.

```
"a" in ensemble_b
True
```

Structures de données :

Les ensembles (Union - Intersection - Différence**)**

- Les ensembles sont représentés par un type de données permettant d'effectuer des opérations telles que l'union, l'intersection et la différence grâce à des méthodes intégrées.

- L'union de deux ensembles peut être effectuée en utilisant l'opérateur | ou la méthode union().

```
ensemble_a = {1, 2, 3, 4}

ensemble_b = set(["a", "b", 7, 3])
```

ou bien

```
union_a_b= ensemble_a | ensemble_b

union_a_b= ensemble_a.union(ensemble_b)

print(union_a_b)
{1, 2, 3, 4, 7, 'a', 'b'}
```

Structures de données :

Les ensembles (Union - **Intersection** - Différence**)**

- L'intersection de deux ensembles peut être effectuée en utilisant l'opérateur & ou la méthode intersection().

ou bien

```
ensemble_a = {1, 2, 3, 4}

ensemble_b = set(["a", "b", 7, 3])

intersection_a_b= ensemble_a & ensemble_b

intersection_a_b= ensemble_a.intersection(ensemble_b)

print(intersection_a_b)
{3}
```

Structures de données :

Les ensembles (Union - Intersection - **Différence)**

- La différence entre deux ensembles peut être effectuée en utilisant l'opérateur **-** ou la méthode difference().

```
ensemble_a = {1, 2, 3, 4}

ensemble_b = set(["a", "b", 7, 3])

difference_a_b= ensemble_a - ensemble_b
```

ou bien

```
difference_a_b= ensemble_a.difference(ensemble_b)

print(difference_a_b)
{1, 2, 4}
```

❑ Applications

- **Application 1 :** Crée une liste de fruits, puis ajoute un fruit à la liste et affiche la liste complète.

```
fruits = ["pomme", "banane", "cerise"]
fruits.append("orange")
print(fruits)
```

- **Application 2 :** Crée un dictionnaire avec des informations de base (nom, âge, ville) et affiche chaque information sur une ligne différente.

```
personne = {"nom": "Alice", "âge": 25, "ville": "Paris"}
for cle, valeur in personne.items():
    print(f"{cle} : {valeur}")
```

❑ Boucles for et while

- **Concepts :** Boucles pour répétition de tâches

Exemples :

```
# Déclaration d'une liste de couleurs
couleurs = ['rouge', 'bleu', 'vert', 'jaune']
# Boucle pour itérer sur chaque couleur de la liste
for couleur in couleurs:
    # Affichage de chaque couleur
    print(couleur)
```

- ✓ Boucle for : Utile pour parcourir des séquences (listes, chaînes, etc.).
- ✓ Boucle while : Permet une répétition basée sur une condition.

❑ Les fonctions

- Une fonction en Python est un bloc de code que l'on peut réutiliser et exécuter à tout moment. Elle prend des entrées (appelées paramètres ou arguments), effectue des actions spécifiques, puis retourne un résultat.

- Nous avons déjà utilisé plusieurs fonctions prédéfinies, comme print(), input(), et range(). Ce sont des fonctions intégrées à Python (ou built-in functions). En plus de ces fonctions, nous avons également la possibilité de créer nos propres fonctions pour résoudre des problèmes spécifiques à notre programme !

❑ Les fonctions

- Les fonctions sont des blocs de code qui accomplissent une tâche spécifique, permettant de rendre un programme modulaire en le divisant en parties plus simples. Elles évitent la répétition du code, facilitent la compréhension, la maintenance et rendent le programme plus lisible et réutilisable.

❑ Les fonctions

Défintion de fonction

Syntaxe générale :

```
def nom_de_la_fonction(parametre1, parametre2, ...):
    # Corps de la fonction
    # Opérations à effectuer
    return resultat
```

def : C'est le mot-clé utilisé pour définir une fonction.

nom_de_fonction : Le nom que vous donnez à la fonction. Il doit être descriptif pour refléter l'action qu'elle accomplit.

parametre1, parametre2 : Ce sont des valeurs que la fonction peut recevoir (ces paramètres sont optionnels). Ce sont des variables locales à la fonction.

return : Ce mot-clé est utilisé pour renvoyer une valeur à l'endroit où la fonction a été appelée. Il est facultatif. Si une fonction ne retourne rien, elle renvoie implicitement None.

❑ Les fonctions

Appel de la fonction

L'appel d'une fonction consiste à exécuter la fonction en utilisant son nom et en lui fournissant les arguments nécessaires, si elle en nécessite, afin qu'elle effectue son opération et, si applicable, retourne un résultat.

Syntaxe générale :

```
nom_de_fonction()   # Appel d'une fonction sans paramètres
```

Si la fonction nécessite des paramètres, vous devez passer les valeurs appropriées dans les parenthèses :

```
nom_de_fonction(param1, param2,..) # Appel d'une fonction avec des paramètres
```

❑ Les fonctions

Exemple 1

Fonction sans paramètres et sans retour de valeur :

```
def saluer():
    print("Bonjour !")
saluer()
```

```
Bonjour !
```

Exemple 2

Fonction avec des paramètres et sans retour de valeur :

```
def multiplier_et_afficher(a, b):
    produit = a * b
    print("Le produit est:", produit)

multiplier_et_afficher(5, 3)
```

```
Le produit est: 15
```

❑ Les fonctions

Exemple 3

Fonction avec paramètres et retour de valeur:

```
def addition(a, b):
    return a + b

resultat = addition(10, 20)
print(resultat)
```

❑ Les fonctions

```
def somme(*args):
    total = 0
    for num in args:
        total += num
    return total

print(somme(1, 2, 3))
print(somme(10, 5, 12, 2))
```

```
6
29
```

❑ Introduction à la gestion des fichiers avec Python

Cette section montre comment utiliser Python pour manipuler des fichiers texte. Nous apprenons à ouvrir, lire, écrire et fermer des fichiers.

Définitions

- **Fonction open()** : Permet d'ouvrir un fichier et retourne un objet fichier.
- **Mode de lecture (r)** : Mode pour lire un fichier.
- **Mode d'écriture (w)** : Mode pour écrire dans un fichier, écrasant le contenu existant.
- **Mode d'ajout (a)** : Ajoute du contenu à la fin du fichier sans écraser les données existantes.

❑ Introduction à la gestion des fichiers avec Python

Exemple

- Lire un fichier :

```
with open("Example1.txt", "r") as file:
    data = file.read()
    print(data) # Affiche le contenu du fichier
```

✓ **open("Example1.txt", "r")** ouvre le fichier en mode lecture.

✓ **file.read()** lit tout le contenu, qui est ensuite affiché avec **print(data)**.

- Écrire dans un fichier

```
with open("Example2.txt", "w") as file:
    file.write("Ceci est la ligne A\n") # Écrit une ligne dans le fichier
    file.write("Ceci est la ligne B\n") # Écrit une autre Ligne
```

✓ **open("Example2.txt", "w")** ouvre le fichier en mode écriture, créant un nouveau fichier s'il n'existe pas.

✓ **file.write()** ajoute chaque ligne dans le fichier.

❑ Bibliothèques et Pandas

❖ Pandas est une bibliothèque puissante de manipulation de données. Elle est utilisée pour lire, analyser et transformer des données stockées dans divers formats de fichiers, comme les fichiers CSV en utilisant des structures comme les DataFrames.

❖ DataFrame : Structure de données 2D similaire à une feuille de calcul, avec des lignes et des colonnes nommées.

Exemple : Charger un fichier CSV dans un DataFrame

```
import pandas as pd
    df = pd.read_csv("example.csv")
    print(df.head()) # Affiche Les cing premières lignes
```

❑ Utilisation de NumPy

❖ NumPy : Bibliothèque Python qui permet d'effectuer des calculs numériques avec des tableaux N-dimensionnels (N-dimensional Arrays).

❖ Tableau : Structure de données ordonnée où chaque élément est du même type.

❖ ***Exemple :*** Créer un tableau NumPy et accéder aux éléments

```python
import numpy as np
arr = np.array([1, 2, 3, 4, 5])
print(arr[0])  # Affiche le premier élément
```

✓ **np.array([1, 2, 3, 4, 5]) crée un tableau.**

✓ **arr[0] accède au premier élément, qui est 1.**

❑ Opérations de base avec Pandas

filtrage des données dans un DataFrame et enregistrement des résultats dans un nouveau fichier CSV.

Exemple : Filtrer les données et sauvegarder dans un fichier CSV :

```
df_filtered = df[df['Year'] > 1979]  # Filtrer les années après 1979
df_filtered.to_csv("filtered_data.csv", index=False)  # Sauvegarder le fichier filtré
```

- ✓ **df['Year'] > 1979** crée une condition pour filtrer les années après 1979.
- ✓ **df.to_csv("filtered_data.csv")** sauvegarde le **DataFrame** filtré dans un nouveau fichier CSV.

❑ Introduction à l'utilisation de NumPy en 1D

Nous explorons les opérations de base de NumPy sur les tableaux unidimensionnels.

Exemple :

```
u = np.array([1, 0])
v = np.array([0, 1])
z = u + v  # Addition des vecteurs
print(z)  # Affiche [1 1]
```

❑ Tableaux NumPy en 2D

Utilisation de NumPy pour manipuler des tableaux 2D et effectuer des opérations matricielles.

np.dot(X, Y) effectue une multiplication matricielle entre X et Y.

```
X = np.array([[1, 2], [3, 4]])
Y = np.array([[2, 0], [1, 2]])
Z = np.dot(X, Y)  # Multiplication de matrices
print(Z)
```

❑ Introduction aux APIs

Les APIs permettent la communication entre différentes applications et sources de données, offrant un accès aux informations disponibles en ligne.

Définitions

- ✓ API : Interface qui permet à deux logiciels de communiquer.
- ✓ REST : Style d'API basé sur des requêtes HTTP pour accéder à des ressources.

Exemple :

```
import requests
response = requests.get("https://api.coingecko.com/api/v3/simple/price?ids=bitcoin&vs_currencies=usd")
print(response.json())  # Affiche le prix actuel du Bitcoin en USD
```

- ✓ requests.get() envoie une requête GET à l'API CoinGecko
- ✓ response.json() retourne les données en format JSON.

❑ Introduction aux APIs

Exemple : Cet exemple illustre l'utilisation de l'API Watson pour convertir un fichier audio en texte et traduire ce texte dans une autre langue.

```
from ibm_watson import SpeechToTextV1
speech_to_text = SpeechToTextV1(url="URL_ENDPOINT", iam_apikey="YOUR_API_KEY")
with open('audio.wav', 'rb') as audio_file:
    response = speech_to_text.recognize(audio=audio_file, content_type='audio/wav')
    print(response.get_result())
```

✓ Le fichier audio est transmis au service Watson, qui renvoie une transcription textuelle.

❑ Protocole HTTP et API REST

Le protocole HTTP permet la communication sur le web et est la base des API REST.

Définitions

- ✓ HTTP : Protocole utilisé pour transférer des informations sur le web.
- ✓ URL : Adresse web permettant d'accéder à une ressource spécifique.

Exemple :

```
import requests
response = requests.get("https://jsonplaceholder.typicode.com/posts")
print(response.status_code)
```

- ✓ **requests.get()** envoie une requête GET
- ✓ **status_code** vérifie si la requête a réussi (code 200).

❑ HTML pour le Web Scraping

HTML est le langage de balisage des pages web. Comprendre sa structure nous permet d'extraire des données de pages web.

Définitions

- ✓ Tag HTML : Un élément de HTML qui contient des informations pour afficher du contenu web.

Exemple : Extraire des titres d'une page web

```
from bs4 import BeautifulSoup
html = "<html><head><title>Page Title</title></head></html>"
soup = BeautifulSoup(html, "html.parser")
print(soup.title.string)  # Affiche "Page Title"
```

❑ différents formats de fichiers

Python permet de lire et manipuler des fichiers de différents formats comme CSV, JSON, et XML.

```
import json
with open("data.json", "r") as file:
    data = json.load(file)
    print(data)  # Affiche le contenu JSON sous forme de dictionnaire Python
```

✓ **json.load(file)** lit le fichier JSON et le convertit en dictionnaire Python**.**